

Executive Summary

Deploying a full-stack JavaScript app (static frontend + Express backend) to Vercel requires reorganizing the code for Vercel's serverless model and Git-based workflow. In practice, we: (1) **Prepare the code** by removing hardcoded `localhost:3000` URLs in the frontend, relocating static assets into a `public/` folder, and converting Express routes into Vercel Functions (files under `/api`). We eliminate any `app.listen(...)` and instead *export* request handlers as serverless functions (Vercel auto-scales them). (2) **Handle environment variables** by moving secrets from `.env` into Vercel's Dashboard (e.g. `CLOUDINARY_API_KEY`, etc). (3) **Push to GitHub** and connect the repo in Vercel ("Import Project"), setting the **Root Directory**, **Framework Preset: Other**, and defining Build & Output settings and Env vars `["24+L1781-L1792"]`. Finally, we **test and debug**, checking the deployed site (endpoints, images, CORS). This report gives a detailed checklist and code snippets for each step, plus diagrams and tables comparing deployment options.

1. Analyze Project Structure – Files to Change

Review the repo layout. In our example, we have:

- `Client/` – static HTML/CSS/JS frontend (plain files, not a React build).
- `Img/` – image assets (referenced by client).
- `Server/` (and also root `index.js`) – Express backend(s) using `app.listen`.
- `.env` – clouinary keys (sensitive).

Files needing edits:

- **Frontend JS:** `Client/main.js`, `Client/product.js`, etc. They contain `fetch('http://localhost:3000/...')` or hardcoded base URLs. These must be changed to use relative calls (e.g. `fetch('/api/chotot?...')` or `const API_PROXY = window.location.origin`).
- **Image paths:** Client code references images at `../Img/...`. We should either move `Img/` into a public directory or update paths (see step 3).
- **Server files:** Both `Server/index.js` and root `index.js` use `app.listen(...)`. For Vercel, we **remove** `app.listen` and export handlers instead. We will break out each route (e.g. `/api/chotot`, `/upload`) into its own file under `api/`.
- **.env:** Remove sensitive keys from code. We will **not** push `.env` to GitHub, but instead set those values in Vercel's Environment Variables.

Making these changes ensures no hardcoded localhost calls, and that back-end code fits Vercel's functions model. Official docs note that when deploying an Express app to Vercel, you either export the app (`module.exports = app`) or use `app.listen` inside the serverless function entrypoint `["7+L1773-L1781"]` `["5+L1860-L1864"]`. We prefer exporting or writing standalone functions. Also, note that Vercel *ignores* `express.static()`: static assets must live in `public/` `["5+L1858-L1864"]`.

2. Frontend: Remove `localhost` and Fix Fetch Calls

In each client JS file with API calls, replace hardcoded base URLs: e.g.

```
- const API_BASE = "http://localhost:3000";
+ const API_BASE = ""; // or use window.location.origin

- fetch(`${API_BASE}/api/chotot?limit=200`)
+ fetch(`/api/chotot?limit=200`)
```

Or simply use relative paths:

```
// Example replacement in Client/main.js or product.js
async function loadData() {
  const res = await fetch('/api/chotot?limit=200');
  const data = await res.json();
  // ...
}
```

This ensures the app calls the same origin, whether local or on Vercel. Similarly, update any code doing `new URL('http://localhost:3000')` to use `window.location.origin` or no host at all.

Also update image URLs if needed (see next section). Test locally to verify no more `localhost:3000` references remain.

3. Static Assets & Folder Structure

Move all static frontend files and images into Vercel's expected static directory. A common pattern: create a top-level `public/` directory. Inside it:

- Copy all `Client/` **HTML/CSS/JS files** into `public/`.
- Copy `Img/` (or rename it to `public/Img/` or `public/images/`).
- Update references if necessary: e.g. `` (leading slash means from `public/` root). In JS/HTML code, change `src="../Img/qc1.jpg"` to `src="/Img/qc1.jpg"`.

Vercel will serve files in `public/` at root URLs (see *Serving static assets* [5†L1860-L1864]). If you prefer, you can also treat the original `Client/` folder as the output: in the Vercel dashboard, set "Output Directory" to `Client`. But using `public/` is simplest and follows Vercel's convention (it automatically serves `public/index.html` etc).

Remember to update any client-side `<link>` or `<script>` tags: if `index.html` was in `Client/`, and you moved it to `public/`, then references like `<script src="../main.js">` should remain (path unchanged), but `` needs the leading slash as above.

Resulting structure suggestion:

```
project-root/  
  public/  
    index.html  
    main.js  
    product.js  
    ... (other HTML/CSS/JS files)  
  Img/  
    qc1.jpg  
    banner1.jpg  
    ...  
  api/  
    chotot.js  
    upload.js  
  package.json  
  .env          (not committed)  
  .gitignore  
  README.md
```

Use Mermaid or text diagram if desired (below).

```
graph LR
    subgraph GitHubRepo ["GitHub Repo"]
        Code["Client HTML/CSS/JS", "api/ functions", "package.json", ...]
    end
    subgraph VercelDeployment ["Vercel Deployment"]
        Public["/public (static files)"]
        API["/api (serverless functions)"]
    end
    Code -->|push| GitHubRepo
    GitHubRepo -->|Import/Deploy| VercelDeployment
    Public -->|served via CDN| User
    API -->|runs on-demand| User
```

4. Backend: Converting Express to Vercel Functions

Vercel runs backend code as serverless functions under `/api`. There are two common approaches: export an Express app as a function or write each endpoint as a function. Here we'll convert each route to a separate function file in `api/`.

- **Remove** `app.listen(...)`. In Vercel, you do **not** start a listening server. Instead, each file under `api/` is its own function.
- **Create** `api/chotot.js`: This file handles the chotot proxy logic. For example:

```

// api/chotot.js
import fetch from 'node-fetch'; // if needed or use global fetch
// (If using CommonJS, do: const fetch = require('node-fetch');)

export default async function handler(req, res) {
  try {
    const { adId, page = 1, pages = 1, offset = 0, cg, before_time,
    cacheDuration = 3600000, limit = 200 } = req.query;
    // Build API URL (simplified example)
    let apiUrl;
    if (adId) {
      apiUrl = `https://gateway.chotot.com/v1/public/ad-listing?adId=${
        encodeURIComponent(adId)}`;
    } else {
      apiUrl = `https://gateway.chotot.com/v1/public/ad-listing?page=${page}
      &limit=${Math.min(limit,50)}`;
      if (cg) apiUrl += `&cg=${encodeURIComponent(cg)}`;
      if (offset) apiUrl += `&o=${offset}`;
      if (before_time) apiUrl += `&before_time=${
        encodeURIComponent(before_time)}`;
    }
    console.log('Fetching Chotot API URL:', apiUrl);
    const response = await fetch(apiUrl, { headers: { 'Accept': 'application/
    json' }});
    if (!response.ok) {
      const text = await response.text();
      return res.status(response.status).send(text);
    }
    const data = await response.json();
    return res.json(data);
  } catch (err) {
    console.error('Proxy error', err);
    return res.status(500).json({ error: 'Internal server error', message:
    err.message });
  }
}

```

Note: No `app.get(...)` or `app.listen`. We **export default** an async function `handler(req, res)` that Vercel will invoke for `/api/chotot`. This matches Vercel's docs for Node.js functions. (You can still use Express by doing `const app = express(); ... module.exports = app;` as in their example, but a single function is simpler here.)

- **Create other functions as needed:** For example, `api/upload.js` to handle the POST `/upload` (Cloudinary) route. Copy the logic from the old Express route into an exported function. Ensure to `await cloudinary.uploader.upload_stream(...)` properly.

- **Ensure** `api/` **directory is at project root** (next to `public/`). Vercel will automatically make `/api/<file>.js` available. No `vercel.json` is strictly needed for basic setups.
- **Dependencies:** Ensure any NPM packages (express, node-fetch, cors, multer, cloudinary, etc.) are listed in `package.json`. Vercel will install them during build.

5. Environment Variables (Vercel Dashboard)

Move secrets from `.env` to Vercel's environment settings:

- Open Vercel Dashboard → *Project* → *Settings* → *Environment Variables*.
- Add entries matching your keys. For example:
- `CLOUDINARY_CLOUD_NAME` = e.g. `Root`
- `CLOUDINARY_API_KEY` = e.g. `872691561523823`
- `CLOUDINARY_API_SECRET` = e.g. `Yg9YeZGK2pCmx-ksKV5Iv8XnjJw`

Use **exact variable names** that your code expects (`process.env.CLOUDINARY_API_KEY`, etc). Do **not** commit `.env` to GitHub (add it to `.gitignore`). Vercel's docs explain that environment variables set in the dashboard are available during build and at runtime [\[12†L1715-L1724\]](#). If using Firebase or other services, similarly set their API keys (e.g. `FIREBASE_API_KEY`, `FIREBASE_AUTH_DOMAIN`, etc).

Example `.gitignore` snippet:

```
node_modules/
.env
.vercel/
```

Testing locally: You can run Vercel locally with `vercel dev` (requires Vercel CLI) and it will pick up a local `.env` or you can use `vercel env pull`. But in general, test by pushing commits to GitHub and letting Vercel deploy.

6. Git & GitHub Setup

Initialize and push the project to GitHub. Recommended branch: `main`. For example:

```
git init -b main
git add .
git commit -m "Prepare project for Vercel deployment"
git remote add origin https://github.com/<your-username>/<your-repo>.git
git push -u origin main
```

This matches GitHub's guide (first commit on `main` branch) [\[38†L175-L183\]](#) [\[38†L188-L197\]](#). Make sure `.gitignore` excludes `node_modules` and your local `.env`.

In GitHub, your code should show the structure: `public/`, `api/`, etc.

7. Vercel Project Import & Settings

1. Log in to Vercel and click **New Project → Import Git Repository**.
2. Select your GitHub repo.
3. In the setup page:
4. **Name:** your choice (or default).
5. **Root Directory:** usually the project root (leave blank unless your code is inside a subfolder).
6. **Framework Preset:** Choose **Other** (since this is vanilla JS).
7. **Build & Output Settings:**
 - If you have a build step (e.g. a React/Vite app), set the build command and output folder. For a plain static site, you can leave build command blank and output as `public/`.
 - Otherwise, you may not need a build step if using only static files and simple functions.
8. **Environment Variables:** Re-enter the keys from step 5 (they can also be set later).
9. Click **Deploy**.

Vercel will clone your GitHub repo and start a deployment. The **Deployments** log will show any errors. You should see lines like “Static files: X, Serverless Functions: Y” if using `vercel.json` or automatically detected (like in the StackOverflow example [【18†L259-L268】](#)).

Once deployed, Vercel provides a live URL (e.g. `https://your-project.vercel.app`). All `/public` files are served at root, and your functions at `/api/` .

8. Testing & Troubleshooting

- **Check Home Page:** Visiting `https://<your-project>.vercel.app/` should load `index.html` . If you see a 404 or blank page, ensure your output directory is correct. For client-side routing (if any), you might need a `vercel.json` rewrite rule (for React Router, etc) – but for static HTML, it shouldn't be needed.
- **Check Images:** Make sure images load. If they're 404, verify `public/Img/...` and that `` is correct. Remember Vercel serves `public/` at the root.
- **Test API:** Open `https://<your-project>.vercel.app/api/chotot?limit=5` . You should get JSON from the Chotot proxy. If you get 404, confirm `api/chotot.js` exists and has `export default` . If you see CORS errors, note that calling `/api` from your own domain should not trigger CORS. If your React or JS code was still calling `localhost` , it will still fail – double-check step 2 changes.
- **Console/Logs:** In Vercel Dashboard, under **Deployments**, click a deployment and view the **Logs**. Functions logs (e.g. `console.log` from your code) will appear. This can help debug issues.
- **CORS Issues:** If you previously had `cors()` in Express, it's usually not needed if frontend and backend are on same domain. If you still see “No 'Access-Control-Allow-Origin'” errors, verify that your client is hitting the Vercel domain (not localhost) and that the function doesn't restrict origins.

- **404 Debugging:** Vercel's docs on [Debugging 404 Errors](#) suggest checking routes and casing. In our setup, verify that file names in `/api/` exactly match the URLs (e.g. `api/chotot.js` → `/api/chotot`).

Common symptom → fix table:

Symptom	Likely Cause	Fix
Blank or 404 at root URL	Wrong output folder or missing index.html	Ensure <code>public/index.html</code> exists; set output directory to <code>public/</code> in settings
GET <code>https://.../api/chotot</code> returns 404	Function file missing or misnamed	Check <code>api/chotot.js</code> is present and exported correctly; redeploy
Frontend still calls localhost	Forgot to update fetch URLs	Replace <code>http://localhost:3000</code> with relative (<code>/api/...</code>)
Images not loading (404)	Static path incorrect	Move images to <code>public/Img</code> ; use <code></code>
CORS errors in browser console	Calling different domain/port	Ensure calls use same origin (no <code>localhost</code>), or configure CORS in function.
Internal server error on function	Error in backend code	Check Vercel function logs; missing env var, or code thrown an exception.
Build timeouts (>45s on Hobby plan)	Long install/build	Remove unused packages, increase plan, or use faster bundler

9. Deployment Options: Vercel Functions vs Separate Backend

Criterion	Vercel (Serverless)	Separate Backend (e.g. Heroku/Render)
Scaling	Auto-scales to zero and up on demand (no idle cost) [7†L1773-L1781]. Cold-start latency possible.	May require dynos/instances running; can scale but idle cost.
Persistent State	<i>Stateless functions.</i> No local disk or memory between calls [36†L39-L47] [34†L328-L336]. Cannot maintain open sockets.	Can maintain in-memory cache, open DB connections, run workers.
Execution Time	Limited (60s on Hobby/Pro, up to 15min Enterprise) [36†L60-L64]. Not for long tasks.	Typically no strict timeouts (depends on platform). Suitable for long jobs.
Always-on Processes	Not supported (no background jobs). Supports only cron-like Serverless functions.	Supported. You can run workers, WebSocket servers, schedulers.
WebSocket Support	No native WebSocket. Needs external service or Edge, since HTTP functions close after response.	Yes (WebSockets, custom TCP) as long as server supports it.
DevOps Overhead	Minimal: zero server management. Connect GitHub and code, Vercel handles infra [24†L1781-L1792].	More to manage (runtime, scaling, domain). Often still easy with PaaS.
Pricing	Free tier generous for front-end; pay for function invocations, bandwidth.	Varies by platform; may have free tier (Railway, Render) or paid plans.

Criterion	Vercel (Serverless)	Separate Backend (e.g. Heroku/Render)
Best Use	Frontend + light backend/API without state. Static sites, webhooks, quick APIs.	Stateful apps, real-time, heavy processing, legacy setups.

According to Vercel's own docs, Express apps can be deployed but become single functions [\[5†L1868-L1872\]](#). Community and blog sources note Vercel is ideal *only if your app is stateless*. For persistent storage or long processes, costs or complexity grow [\[36†L39-L47\]](#) [\[36†L60-L64\]](#). Alternatives like Heroku/Render allow regular servers (background tasks, databases) but involve traditional scaling management.

10. Reusable Checklist Template

A **checklist** for similar deployments might look like:

- **[] Initial Code Cleanup:** Remove localhost URLs, prepare code for production.
- **[] Static Assets:** Consolidate client code and assets into `public/`; adjust paths (no more `../img`).
- **[] API Conversion:** Create `/api/*.js` files, move Express route logic there. Remove any `app.listen`.
- **[] Environment Setup:** Add any secret keys to `.env.local` (for local dev) and to Vercel Dashboard (for prod). Remove secrets from source code.
- **[] Git Setup:** Ensure `.gitignore` excludes `/node_modules` and `.env`. Push code to GitHub `main`.
- **[] Vercel Project:** Import the GitHub repo in Vercel. Configure project: root dir, build command (if any), output directory (`public/`), and add environment variables.
- **[] Deploy & Test:** Trigger a deployment, then verify site, API endpoints, and console logs. Fix errors (404s, CORS, missing files) and redeploy.
- **[] Troubleshooting:** If issues persist, check the logs, ensure function names match routes, and use Vercel docs or support for debugging guidance.

By following these steps and using the code snippets above, a full-stack JS app can be smoothly transitioned to Vercel with GitHub integration. The official guides on Express on Vercel and GitHub importing [\[7†L1773-L1779\]](#) [\[38†L175-L183\]](#) provide the underlying patterns for this process.